Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Project Report

on

"Hospital Management System"

[Code No.: COMP 207]

(For partial fulfillment of Year II / Semester II in Computer Science)

Submitted by

Aayush Aacharya (Roll no. 1)

Aayush Adhikari (Roll no. 2)

Aarjan Adhikari (Roll no. 3)

Krish Ghimire (Roll no. 24)

Submitted to

Mr. Suman Shrestha

Department of Computer Science and Engineering

July 27, 2026

Bona fide Certificate

This project work on

"Hospital Management System"

is the bona fide work of

Aayush Aacharya (Roll no. 1), Aayush Adhikari (Roll no. 2),
Krish Ghimire (Roll no. 24), Aarjan Adhikarii (Roll no. 3)

who carried out the project work under my supervision.

Project Supervisor

Dr. Rabindra Bista
Lecturer
Department of Computer Science and Engineering

Acknowledgements

We express our sincere gratitude to all who contributed to the successful completion of this project.

First, we would like to thank our project supervisor for their valuable guidance, continuous support, and constructive feedback throughout the development of this project. Their insights and encouragement played a crucial role in shaping both the technical and conceptual aspects of our work.

We are also grateful to the faculty members of our department for providing the necessary resources and academic environment that supported our learning and project development.

We would like to acknowledge our teammates for their cooperation, dedication, and collaborative efforts. Working together as a team helped us overcome challenges and successfully implement the project objectives.

Finally, we extend our gratitude to our friends and family for their constant encouragement and moral support throughout the course of this project.

Abstract

The growing complexity of healthcare delivery has increased the need for efficient, secure, and integrated hospital information systems. This project presents the design and development of a web-based **Hospital Management System (HMS)** that automates core clinical and administrative operations and connects the different people involved in patient care through a single platform. The system provides secure user authentication using **JSON Web Tokens (JWT)** with refresh-token rotation, and implements **role-based access control** across five distinct roles patient, doctor, pharmacist, lab assistant, and administrator so that each user interacts only with the features appropriate to them. Patients can register, book and manage appointments, view prescriptions and lab results, and settle bills; doctors manage their availability, consult patients, issue prescriptions, order lab tests, and refer patients to colleagues; lab assistants process test requests; pharmacists manage a location-aware medicine inventory; and administrators manage staff, billing, rooms, and system-wide records. The application is built using **React** with **Tailwind CSS** on the frontend, and a **Node.js** and **Express** backend integrated with a **MySQL** relational database through the **Sequelize** ORM. Beyond in-application alerts, the system delivers **outbound email and SMS notifications**, along with password-reset and email-verification flows, so that patients stay informed without needing to log in. By reducing manual paperwork, preventing double-booking, and centralising clinical records, the proposed system improves the efficiency and reliability of hospital operations and provides a robust foundation for future enhancements.

Keywords: Hospital Management System, React, Node.js, Express, MySQL, Sequelize, JWT Authentication, Role-Based Access Control, RESTful API, Full-Stack Development.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Hospitals coordinate a large amount of information every day: patient records, appointments, prescriptions, laboratory tests, billing, and pharmacy stock. When these are handled manually or through disconnected tools, the result is duplicated effort, long waiting times, lost records, and a higher chance of error. The adoption of integrated digital systems has become essential to improve accuracy, accessibility, and overall efficiency of healthcare delivery.

This project focuses on the development of a **Hospital Management System (HMS)** using modern web technologies. The system is designed to automate core hospital operations—user and staff management, appointment scheduling, prescriptions, laboratory workflows, referrals, billing, pharmacy inventory, and notifications—while ensuring security, scalability, and ease of use. The application follows a full-stack approach with a clear separation between frontend and backend services communicating over a RESTful API.

1.1 Background

In recent years, the digital transformation of healthcare has significantly changed how hospitals manage information. Institutions are increasingly moving from paper-based and semi-automated processes to fully web-based platforms that support real-time access, centralised data storage, and remote usability. The widespread adoption of RESTful APIs, relational databases, and single-page application frameworks has enabled healthcare providers to handle large volumes of structured data more reliably. Technologies such as React, Node.js, Express, and MySQL have become popular for such systems because of their maturity, strong data-integrity guarantees, and ability to support dynamic, user-centric applications.

Despite these advances, many hospital systems still face limitations. Traditional systems often rely on rigid architectures or legacy databases that are difficult to scale and

maintain. They may lack proper security mechanisms, leading to vulnerabilities in authentication, authorisation, and access control—an especially serious concern given the sensitivity of medical data. Older solutions frequently provide limited support for role separation, inefficient scheduling that permits double-booking, and poor interfaces that reduce usability for both patients and staff. Communication is often one-directional: a patient may have no way of learning about an appointment or a ready lab result unless they physically visit or telephone the hospital.

The significance of a modern hospital management solution lies in its ability to overcome these limitations through a modular architecture, secure communication, a well-designed relational schema, and multi-channel notifications. By emphasising separation of concerns, secure API-driven communication, and consistent data modelling, such a system improves both performance and maintainability, and lays a foundation for future integration with other digital health platforms.

1.2 Objectives

The main objectives of this project are:

- To design and develop a web-based **Hospital Management System** using React, Node.js/Express, and a MySQL database accessed through the Sequelize ORM.
- To implement secure authentication and **role-based access control** for five user roles (patient, doctor, pharmacist, lab assistant, administrator), including password-reset and email-verification flows.
- To automate core hospital workflows—appointment scheduling with doctor availability, prescriptions, laboratory tests, referrals, billing, and a location-aware pharmacy inventory.
- To keep users informed through multi-channel notifications (in-application, email, and SMS) so that patients receive updates without needing to log in.
- To provide a responsive, role-specific user interface that is intuitive for both patients and hospital staff.

1.3 Motivation and Significance

The motivation behind this project is to gain hands-on experience developing a real-world, full-stack web application while addressing the practical limitations of traditional hospital systems. By focusing on secure backend design, a normalised relational schema,

and reliable communication between services, the project aims to provide a robust and scalable solution for hospital management.

The significance of this system lies in its ability to improve operational efficiency, minimise manual errors, and offer secure, role-based access to clinical and administrative information. It introduces modern development practices such as JWT authentication with refresh-token rotation, transactional relational data handling, and outbound email/SMS integration, making it a valuable learning experience. The implemented system provides appointment booking, prescriptions, lab-test management, referrals, billing, pharmacy inventory, and notifications—demonstrating both academic and professional relevance and setting it apart from simpler record-keeping systems.

Chapter 2

Related Works

Hospital information systems have evolved considerably over time. Several well-known platforms—both open-source and commercial—have influenced the design of modern web-based systems. This section briefly describes some existing hospital management systems that inspired the design of the proposed system.

2.1 Open-Source Hospital Management Systems

OpenMRS

OpenMRS is one of the most widely used open-source medical record platforms in the world, designed to support healthcare delivery in resource-constrained environments. It provides a configurable data model for patients, encounters, and observations, and is built around a modular architecture that can be extended for different clinical needs. Its success demonstrates the effectiveness of a centralised patient record, role-based access, and browser-based accessibility. However, OpenMRS requires a dedicated server environment and technical expertise to install, configure, and maintain, which can be a barrier for small institutions without dedicated IT staff.

HospitalRun

HospitalRun is an open-source hospital information system aimed at small and rural hospitals, with an emphasis on offline-first operation. It covers patient records, scheduling, billing, and inventory. The project highlights the importance of usability and reliable operation in low-connectivity environments, though its feature set and active maintenance vary across releases.

Bahmni / OpenEMR

Bahmni is an open-source distribution that integrates OpenMRS with billing and laboratory modules to form a more complete hospital system, while OpenEMR is a mature electronic health record and practice-management system supporting scheduling, prescriptions, and billing. These systems demonstrate comprehensive, feature-rich management, but they often require complex configuration and are not optimised for lightweight deployment or full-stack learning environments.

2.2 Commercial Hospital Management Systems

Epic and Oracle Health (Cerner)

Epic Systems and Oracle Health (formerly Cerner) are leading commercial electronic health record platforms used by large hospitals and health networks. They provide comprehensive functionality—clinical documentation, scheduling, pharmacy, laboratory, and billing—at enterprise scale. While extremely capable, such systems involve substantial licensing costs, long implementation cycles, and limited customisation, and are generally unsuitable for small institutions or academic study.

MEDITECH

MEDITECH is another established commercial platform offering integrated clinical and administrative modules. It demonstrates structured, professional-grade workflows but, like other commercial systems, carries cost and complexity that make it impractical for small-scale or educational use.

2.3 Comparison with the Proposed System

The proposed Hospital Management System draws conceptual inspiration from the above platforms. However, unlike large-scale enterprise systems, this project focuses on a lightweight and modular architecture using React, Node.js/Express, and MySQL with the Sequelize ORM. It emphasises:

- Role-based portals for patients, doctors, pharmacists, lab assistants, and administrators.
- Secure authentication using JSON Web Tokens with refresh-token rotation, password reset, and email verification.
- RESTful API communication between the frontend and backend.

- A normalised relational schema with enforced foreign-key relationships and cascade rules.
- Multi-channel notifications (in-app, email, and SMS).
- A responsive Single Page Application (SPA) design.

Thus, the proposed system integrates modern web technologies while maintaining simplicity, scalability, and usability suitable for small to medium-sized institutions and for academic study.

2.4 Review of Existing Works

Several studies have explored the development of health information systems to improve data handling, security, and operational efficiency. Effective management systems are widely held to require robust user authentication, data validation, and reporting mechanisms to ensure reliability and data integrity. While these principles established a strong foundation, many early systems relied on monolithic architectures that limited scalability and flexibility.

With the advancement of web technologies, several web-based hospital systems have been proposed using three-tier architectures. Although such systems improved accessibility and reduced manual workload, many did not separate concerns cleanly or provide fine-grained role separation, making maintenance and extension challenging. Open-source projects such as OpenMRS have demonstrated comprehensive clinical features, but they often require complex configuration and are not optimised for lightweight deployment or modern full-stack learning environments.

Comparison with the Current Project. Unlike many earlier systems, the proposed Hospital Management System adopts a modern full-stack approach with a clear layered architecture. It emphasises secure authentication using JWT with refresh-token rotation, five-way role-based access control, a normalised relational schema managed through an ORM, and outbound notifications by email and SMS. These features address the scalability, security, and communication limitations identified in earlier works, within a codebase that remains small enough to understand and extend.

Chapter 3

Design and Implementation

3.1 System Requirement Specifications

The Hospital Management System is designed using a client–server architecture. The backend exposes RESTful APIs under a common base path (`/api/v1/hms`), while the frontend consumes these Application Programming Interfaces (APIs) to render dynamic views. The system supports role-based access, ensuring that patients, doctors, pharmacists, lab assistants, and administrators interact only with authorised features.

3.1.1 Software Requirements

The software required during this project includes:

- Node.js (JavaScript runtime for the backend)
- Express.js (web framework for RESTful APIs)
- MySQL (relational database server)
- Sequelize (Object–Relational Mapping library)
- React with Vite (frontend SPA framework and build tool)
- Tailwind CSS (utility-first styling)
- Web browser (Chrome, Firefox, Edge)
- Code editor (Visual Studio Code)
- JSON Web Tokens, bcrypt (authentication and password hashing)
- Nodemailer and Twilio (outbound email and SMS)

Functional Requirements

Functional requirements define the core features and capabilities of the system. They specify *what the system should do* and should be specific, measurable, and testable. For the Hospital Management System (HMS), the functional requirements include:

- **User Authentication and Authorisation:** Patients can self-register; all users can log in and log out securely. Access tokens are short-lived and refreshed through rotating refresh tokens. Role-based access control ensures each user reaches only authorised features. The system also supports email verification, password reset, and rate limiting on sensitive endpoints.
- **Staff Management:** Administrators can create staff accounts (doctor, pharmacist, lab assistant, administrator), each assigned an auto-generated staff identifier and a temporary password, with an optional welcome email.
- **Appointment Management:** Patients book appointments with doctors based on the doctor's declared weekly availability and one-off time off. Appointments follow a status workflow (pending, confirmed, cancelled, completed).
- **Prescription Management:** Doctors write prescriptions for patients, optionally tied to a specific appointment, listing medicines with dosage, frequency, and duration. Patients can view their prescriptions.
- **Laboratory Management:** Patients or doctors request lab tests; lab assistants process them through a status workflow (requested, in progress, completed, cancelled) and record results, which patients can then view.
- **Referral Management:** A doctor can refer a patient to another doctor, with the referral tracked through its own status workflow.
- **Billing and Invoicing:** Administrators issue invoices with line items; payments (cash, card, insurance, online) are recorded, and invoice status (unpaid, partial, paid, cancelled) is maintained automatically.
- **Pharmacy Inventory:** Pharmacists manage a location-aware medicine catalogue (section, aisle, shelf), including stock levels, reorder thresholds, pricing, and expiry.
- **Notifications:** The system creates in-application notifications and mirrors them to the user's email (and optionally SMS) for appointments, prescriptions, lab tests, referrals, and invoices.
- **API Integration:** The frontend communicates with the backend via RESTful APIs for all data operations.

Non-Functional Requirements

Non-functional requirements describe how the system performs rather than what it does:

- **Performance:** The system should respond to typical requests within 2–3 seconds under normal load, with queries optimised through appropriate indexes.
- **Scalability:** The relational schema and modular services should support a growing number of users and records without major redesign.
- **Security:** Passwords are hashed with bcrypt; JWT access and refresh tokens secure sessions; refresh tokens are stored in HTTP-only cookies and can be revoked; sensitive tokens (reset/verification) are stored only as hashes.
- **Usability:** The interface should be intuitive, responsive, and compatible with major browsers, requiring minimal training.
- **Reliability:** Notification and delivery failures must never break the primary action that triggered them.

3.1.2 Hardware Requirements

The minimum hardware specifications required to run this project are:

- Minimum 4 GB RAM
- Dual-core processor or higher
- Sufficient storage for source code, dependencies, and the database
- A local or networked MySQL server

3.2 System Design

3.2.1 Use Case Diagram

The use case diagram represents the functional requirements of the Hospital Management System. It illustrates the interaction between the different actors and the system, clarifying the major functionalities and the system boundary.

Roles.

- **Patient** — registers, books and manages appointments, views prescriptions and lab results, and pays invoices.

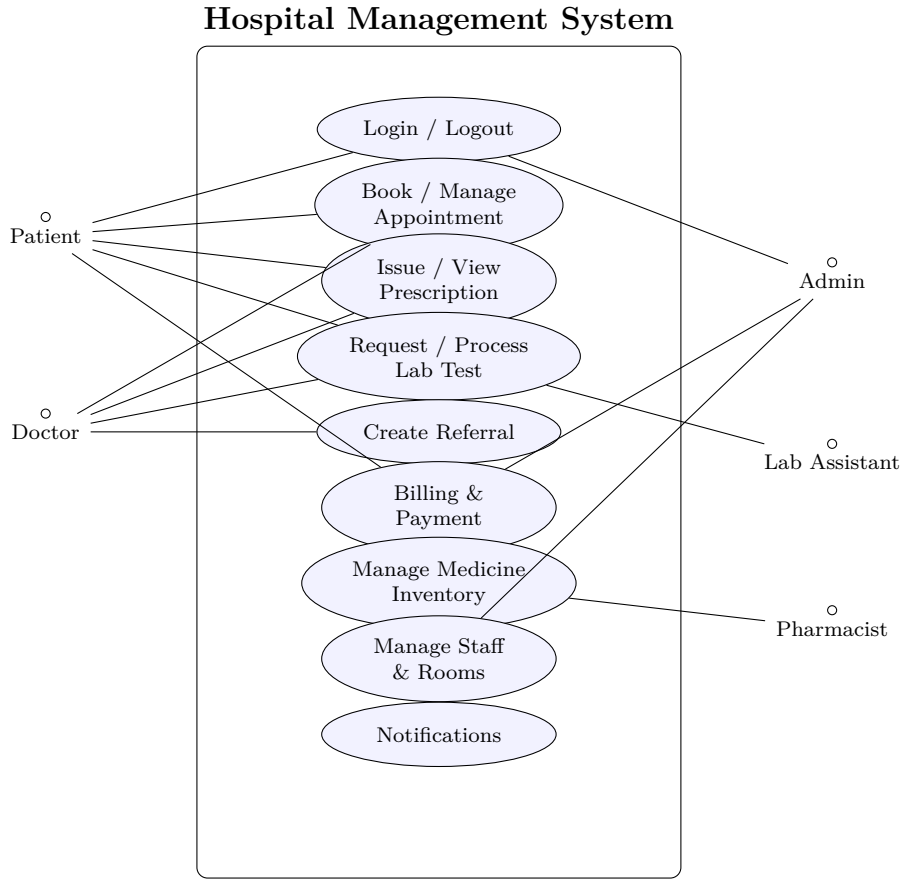

Figure 3.1: Use Case Diagram of the Hospital Management System

- **Doctor** — sets availability, consults patients, issues prescriptions, orders lab tests, and refers patients.
- **Lab Assistant** — processes lab-test requests and records results.
- **Pharmacist** — manages the location-aware medicine inventory.
- **Administrator** — manages staff accounts, rooms, billing, and system-wide records.

3.2.2 Architectural Design

The Hospital Management System is designed using a **layered architecture**, which separates the system into distinct layers, each with specific responsibilities. This modular approach improves maintainability, scalability, and readability.

Presentation Layer. Implemented using React with Vite, this layer manages the user interface and experience. It includes role-specific pages (Login, Register, and the Patient, Doctor, Admin, Lab, and Pharmacist dashboards) and shared components such as `DashboardLayout`, `ProtectedRoute`, `DataTable`, `NotificationBell`, and an authentication context. It communicates with the backend through Axios HTTP requests, attaching the access token and transparently refreshing it when it expires.

Application Layer. Built with Express.js, this layer contains the business logic and middleware. Its key components include:

- **Controllers:** `authController`, `appointmentController`, `availabilityController`, `prescriptionController`, `labController`, `referralController`, `invoiceController`, `notificationController`, `medicineController`, and `staffController`.
- **Routes:** one router per resource, mounted under `/api/v1/hms`.
- **Middleware:** `authenticate` and `authorise` (JWT verification and role checks), `rateLimit`, and centralised error handling.
- **Services:** `tokenService`, `contactService`, `notificationService`, `availabilityService`, `mailer` (Nodemailer), and `sms` (Twilio).

Data Layer. The data layer consists of MySQL with Sequelize models defining schemas, associations, and constraints. It manages persistent data for users, patients, staff, appointments, availability, prescriptions, lab tests, referrals, invoices, notifications, medicines, and verification tokens, enforcing referential integrity and cascade rules.

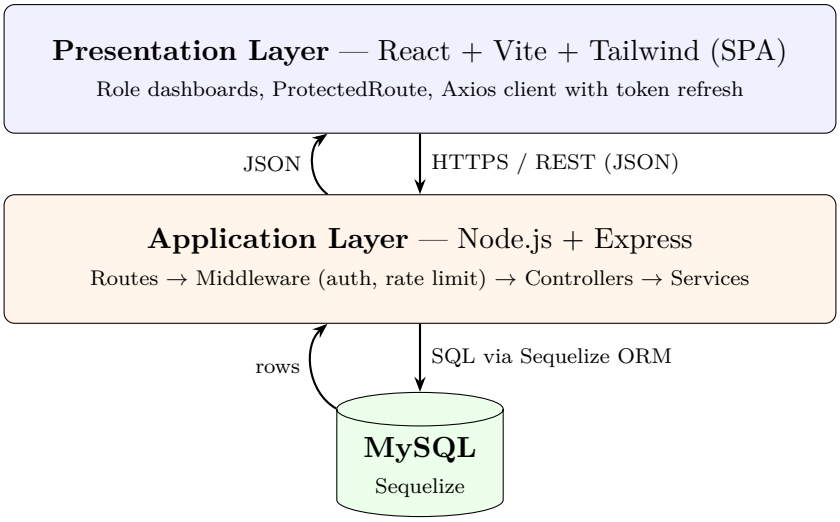

Figure 3.2: Client–Server Layered Architecture of the Hospital Management System

3.2.3 Module or Component Design

The system is divided into several modules, each responsible for specific functionality. This modular structure improves organisation, reusability, and maintainability.

- **Authentication Module:** Handles registration, login, logout, token refresh, password reset, and email verification, backed by hashed credentials and rotating refresh tokens.
- **Appointment Module:** Manages doctor availability, one-off time off, and the booking lifecycle, preventing conflicts against existing bookings.
- **Clinical Module:** Covers prescriptions, laboratory tests, and referrals, linking doctors, patients, and lab assistants.
- **Billing Module:** Manages invoices, line items, payments, and status.
- **Pharmacy Module:** Manages the location-aware medicine catalogue and stock adjustments.
- **Notification Module:** Creates in-app notifications and dispatches email and SMS through the mailer and SMS services.
- **Administration Module:** Manages staff accounts, rooms, and user status.

Each module functions independently but interacts through the RESTful APIs, so the system can be maintained, upgraded, or extended with minimal coupling.

3.2.4 Database or Data Design

The Hospital Management System uses **MySQL** as the primary database and **Sequelize** for defining models and relationships. Unlike a document store, the schema is normalised into related tables with enforced foreign keys and cascade rules. The principal entities are:

- **User** — credentials, role, active status, and refresh-token version; the root of authentication.
- **Patient / StaffProfile** — role-specific profile data linked one-to-one to a User.
- **Appointment**, **DoctorAvailability**, **DoctorTimeOff** — scheduling data linking patients and doctors.
- **Prescription**, **LabTest**, **Referral** — clinical records.
- **Invoice** — billing linked to a patient.
- **Medicine** — the pharmacy catalogue with location and stock.
- **Notification**, **VerificationToken** — alerts and single-use security tokens.

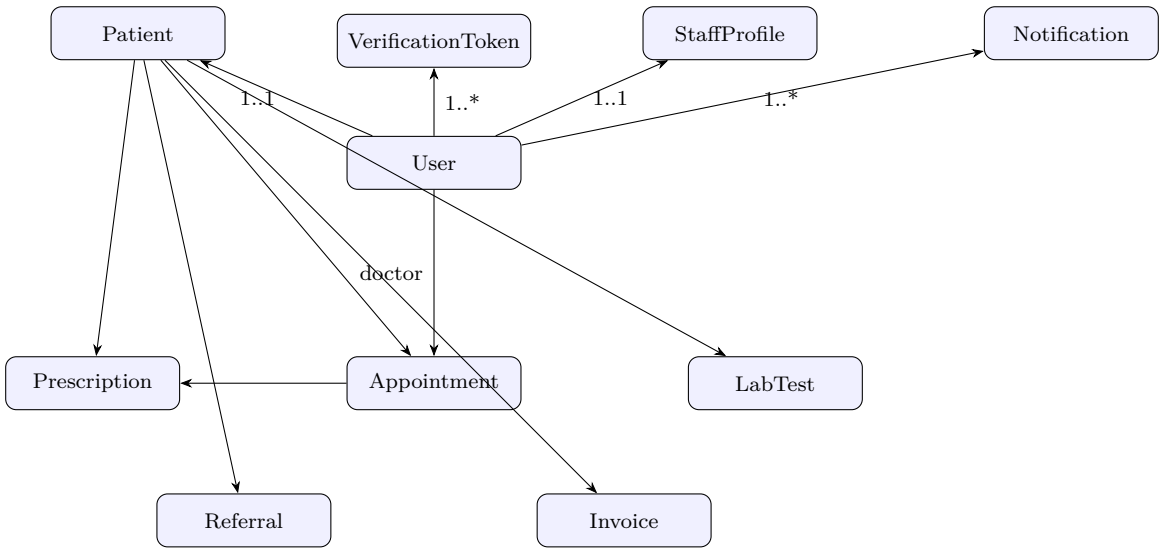

Figure 3.3: Entity–Relationship overview (principal entities and relationships)

3.3 Implementation Details

3.3.1 Software Implementation

The system is implemented as a full-stack web application with a clear separation between the React frontend and the Express backend, communicating over RESTful APIs.

Frontend. React (with Vite) builds a responsive single-page application styled with Tailwind CSS. Axios handles API communication, attaching the JWT access token to each request and transparently refreshing it on expiry. Routing is handled by React Router, with a `ProtectedRoute` component restricting each dashboard to its allowed roles. Authentication state is held in a React context, and toast notifications provide user feedback.

Backend. Node.js with Express implements the RESTful API in a modular structure with separate routes, controllers, middleware, and services. Controllers encapsulate business logic such as booking appointments, issuing prescriptions, processing lab tests, and recording payments. Middleware enforces authentication, role-based authorisation, and rate limiting, and centralised error handling returns consistent responses without leaking internal details.

Database. MySQL stores all persistent data, with Sequelize models defining schemas, validations, associations, and cascade rules. The use of an ORM provides schema consistency, parameterised queries that mitigate SQL injection, and readable association-based data access.

Security. Passwords are hashed with bcrypt. JWT access tokens are short-lived, while refresh tokens are stored in HTTP-only cookies and versioned so they can be revoked (for example, after a password reset). Password-reset and email-verification tokens are single-use and stored only as SHA-256 hashes, and the mail-sending endpoints are rate limited to resist abuse and account enumeration.

3.3.2 Hardware Implementation

The system does not require specialised hardware and runs on standard computing devices. During development and testing it was implemented and evaluated on a personal computer with a dual-core processor, at least 4 GB of RAM, and a local MySQL server. The backend runs on the Node.js runtime, while the frontend runs in a modern web browser, making the system portable across personal computers, institutional servers, and cloud hosts.

3.4 Algorithms and Flowcharts

This section describes the key algorithms used in the system and the logical flow of major operations. These ensure secure authentication, conflict-free scheduling, and proper role-based access control.

Algorithm 1 User Login (JWT)

Start
User submits identifier, password, and role
Validate that all fields are present
Look up the user by identifier; verify the role matches and the account is active
Compare the submitted password with the stored bcrypt hash
if credentials are valid **then**
    Sign a short-lived *access token* and a *refresh token* (with token version)
    Set the refresh token as an HTTP-only cookie
    Return the access token and user profile
else
    Return a generic "Invalid credentials" error
end if
End

Algorithm 2 Access-Token Refresh with Rotation

Start
Read the refresh token from the HTTP-only cookie
Verify the token signature and expiry
Look up the user and compare the token version
if token is valid and version matches **then**
    Issue a new access token
else
    Reject the request and require re-authentication
end if
End

Algorithm 3 Password Reset

Start
User requests a reset with their identifier
Respond identically whether or not the account exists (no enumeration)
if account exists and is active **then**
    Generate a random token; store only its SHA-256 hash with an expiry
    Email the raw token as a reset link
end if
On reset: validate the token hash and expiry; if valid, hash the new password, save it, and *increment the refresh-token version* to revoke all sessions
End

Algorithm 4 Appointment Booking

Start
Patient selects a doctor, date, and time slot
Verify authentication and patient role
Check the doctor's weekly availability and one-off time off for that slot
Check for an existing appointment conflicting with the slot
if slot is available and free **then**
    Create the appointment with status *pending*
    Notify the doctor (in-app and email)
else
    Return an error indicating the slot is unavailable
end if
End

Algorithm 5 Role-Based Access Control

Start
User sends a request to a protected route with an access token
Verify the JWT and load the active user
Extract the user's role
if role is permitted for the route **then**
    Allow the request to proceed
else
    Deny access and return an authorisation error
end if
End

Flowchart — Appointment Booking. Figure 3.4 illustrates the booking process from slot selection through availability checking to record creation and notification.

Flowchart — Authentication. Figure 3.5 shows the login process: credential validation, token generation, and error handling.

3.5 Tools and Technologies Used

This project uses modern web-development tools and technologies to ensure efficiency, scalability, and maintainability, supporting secure, responsive, and modular design.

Layer / Component	**Technologies / Tools Used**
Frontend	React, Vite, Tailwind CSS, Axios, React Router, React Toastify
Backend	Node.js, Express.js
Database	MySQL, Sequelize (ORM)
Authentication	JSON Web Tokens (JWT), bcrypt, HTTP-only cookies
Notifications	Nodemailer (email), Twilio (SMS)
Development Tools	Visual Studio Code, Git, GitHub, Nodemon, ESLint

Table 3.1: Technologies Used in the Hospital Management System

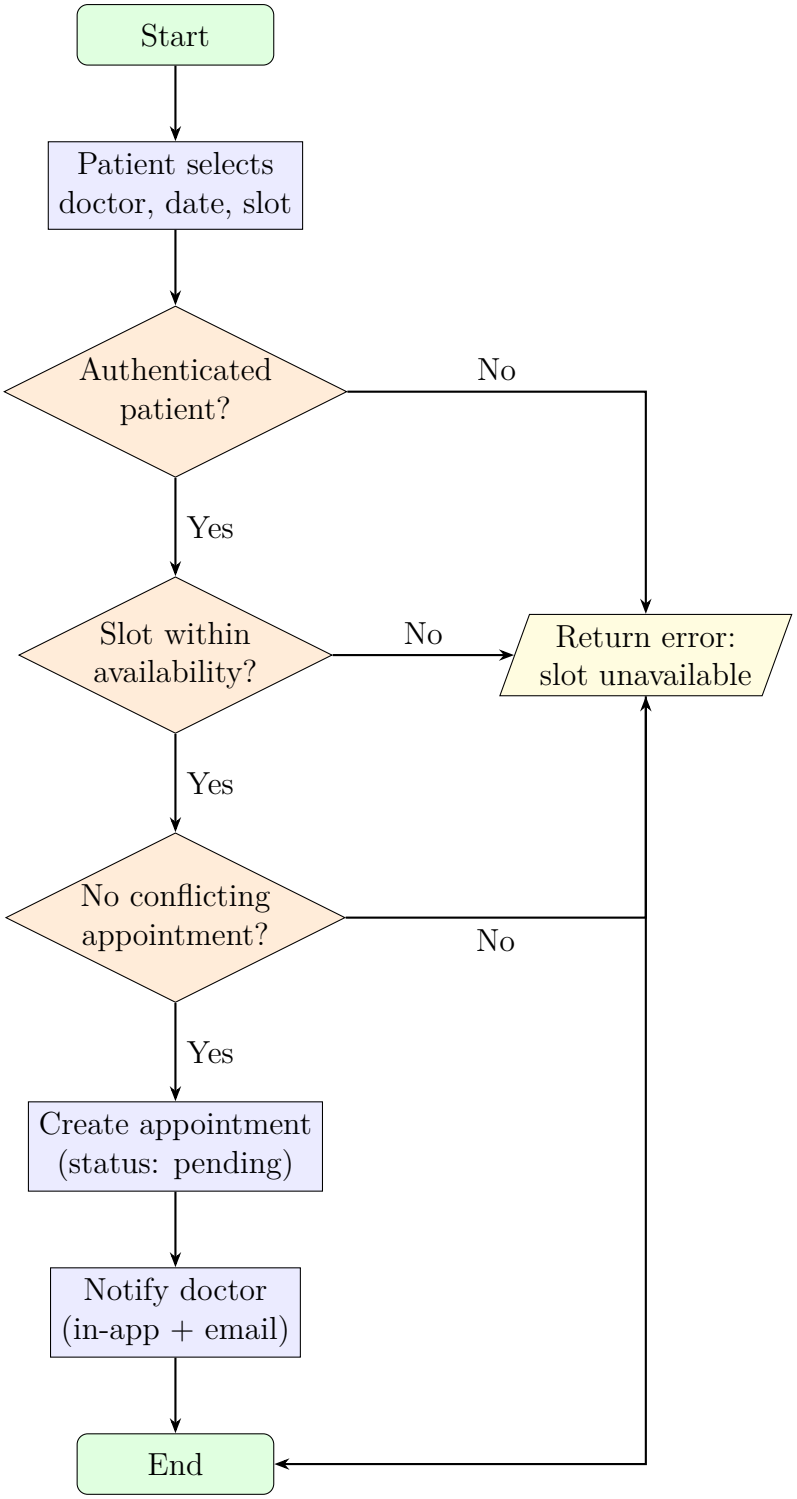

Figure 3.4: Appointment Booking Flowchart

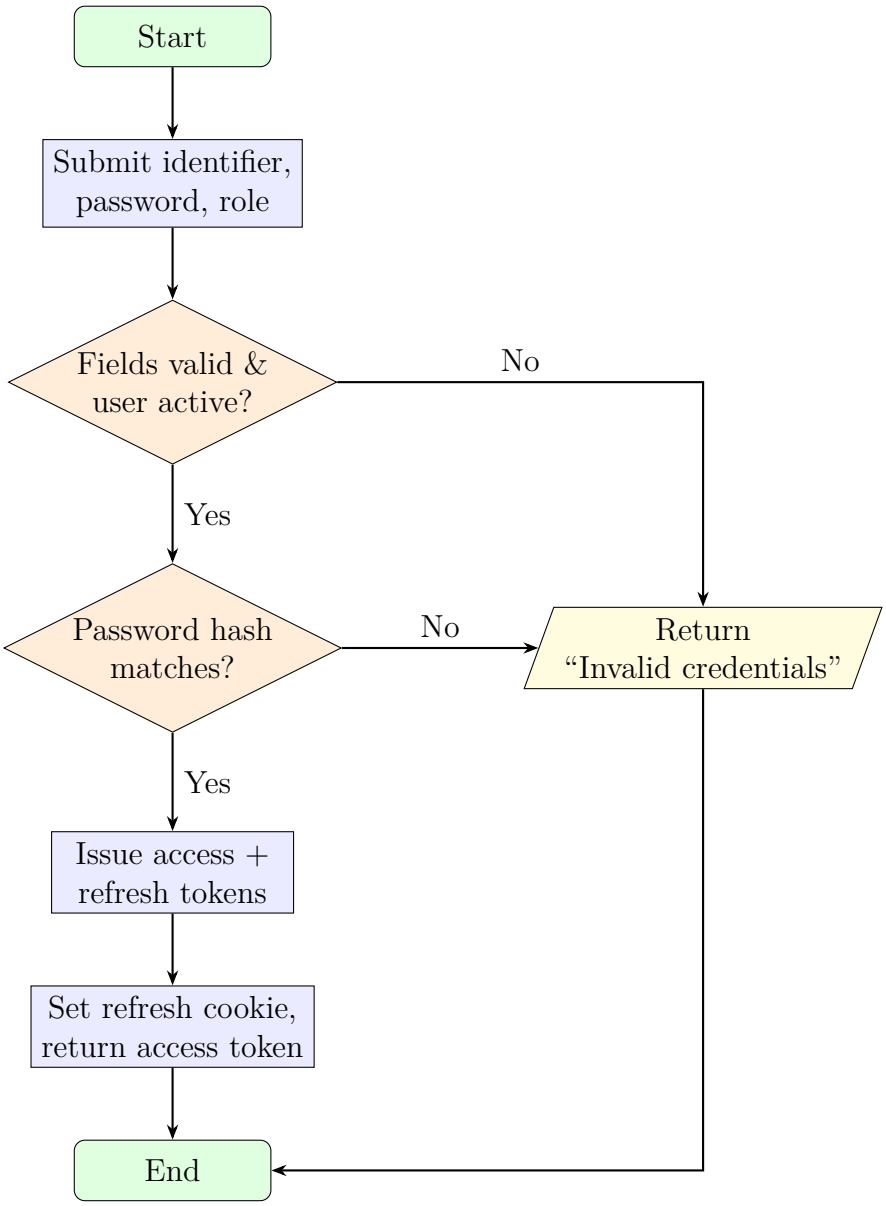

Figure 3.5: User Authentication Flowchart

Chapter 4

Results and Discussion

This chapter presents the results of the implemented Hospital Management System, analyses its behaviour, and compares the outcomes with the objectives stated in Chapter 1. It also discusses the challenges and limitations encountered during development.

4.1 Implemented Features

The system successfully implements the following key features.

Secure Login and Authentication. Users log in through a role-aware form. Authentication is backed by bcrypt-hashed passwords and JWT access tokens, with refresh tokens rotated through HTTP-only cookies. Password-reset and email-verification flows are also provided.

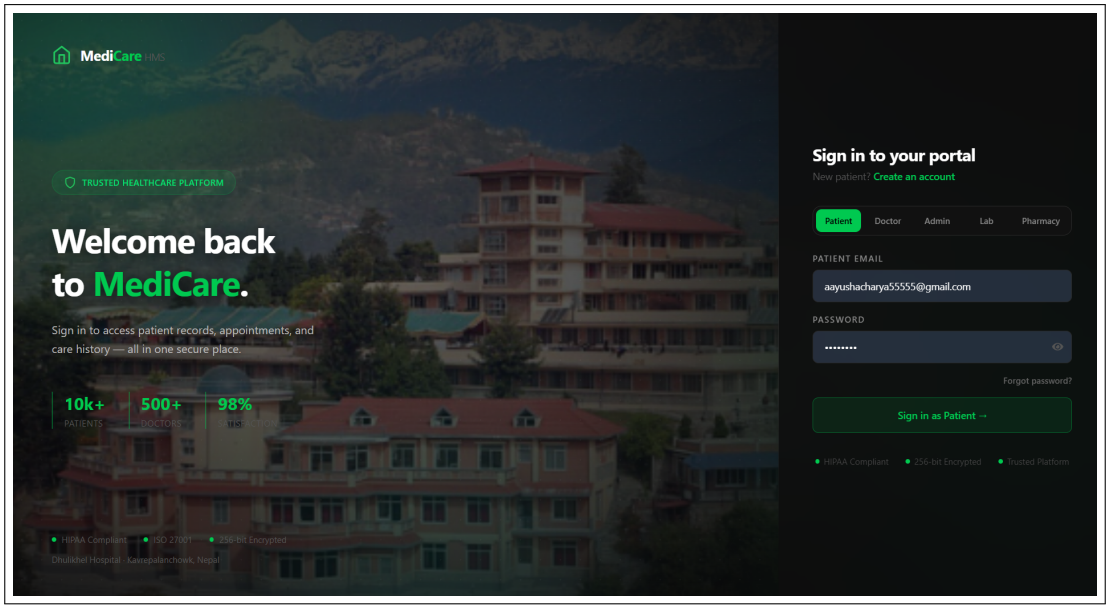

Figure 4.1: Login page — role-aware, secure sign-in

Password Reset and Email Verification. A "Forgot password" flow issues a single-use, hash-stored reset link by email, and new accounts receive a verification link. Where a mail server is not configured, messages are logged to the server console, so the flows remain testable in development.

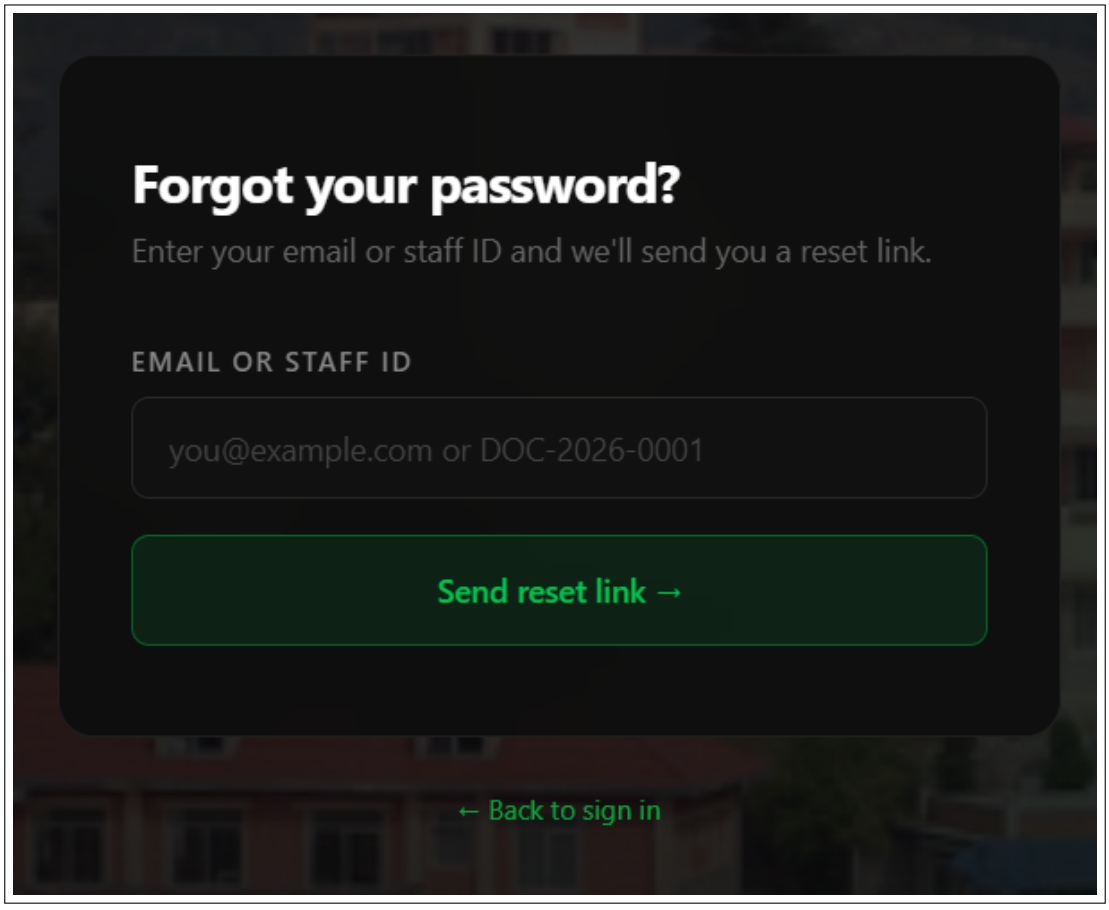

Figure 4.2: Forgot-password page — requests a single-use reset link

Role-Based Dashboards. Each of the five roles has a distinct dashboard. The *patient* dashboard shows appointments, prescriptions, lab tests, and billing; the *doctor* dashboard shows the appointment queue, schedule, patients, and referrals; the *admin* dashboard shows users, staff, billing, lab requests, and room management; the *lab* and *pharmacist* portals show their respective queues and inventory.

Appointment Management. Patients book appointments against a doctor's declared availability, with conflict checking; appointments move through a pending/confirmed/cancelled/complete workflow.

Prescriptions, Lab Tests, and Referrals. Doctors issue prescriptions, order lab tests, and refer patients; lab assistants process test requests and record results, which patients can then view.

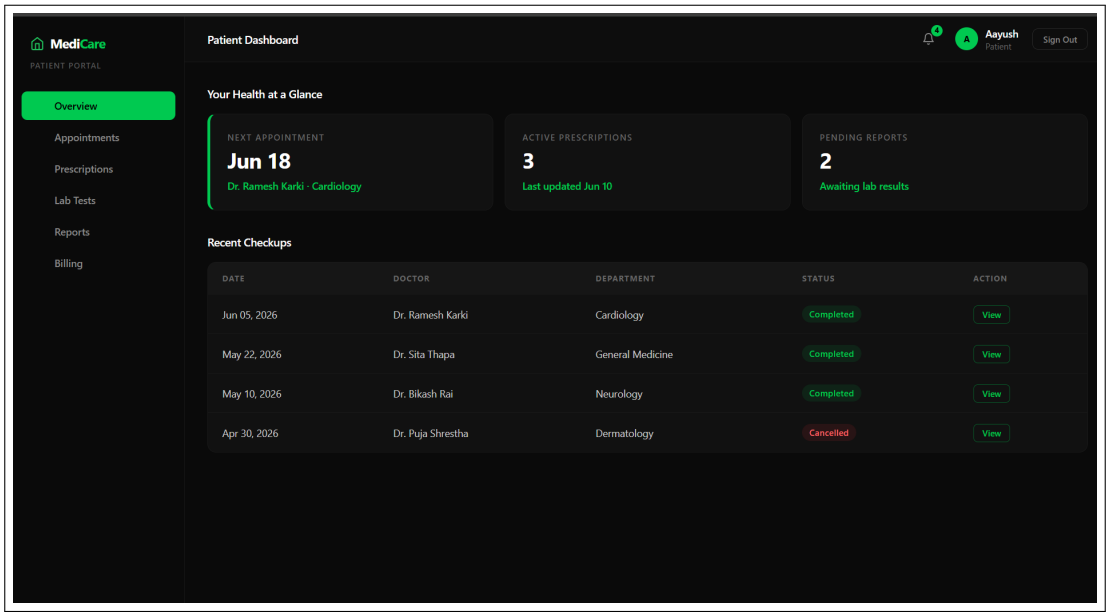

Figure 4.3: Patient dashboard

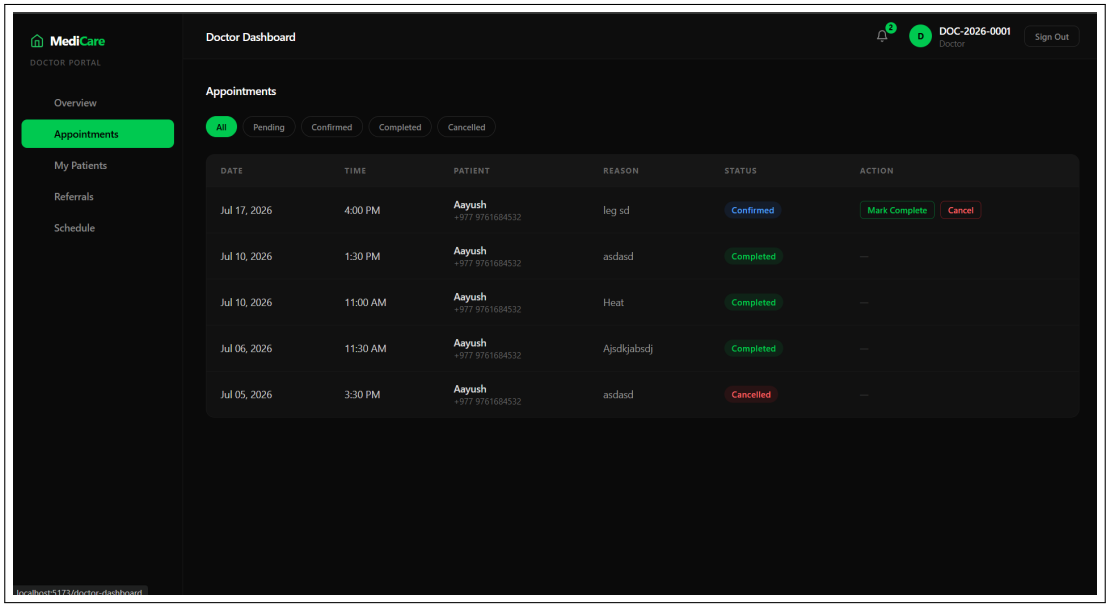

Figure 4.4: Doctor dashboard

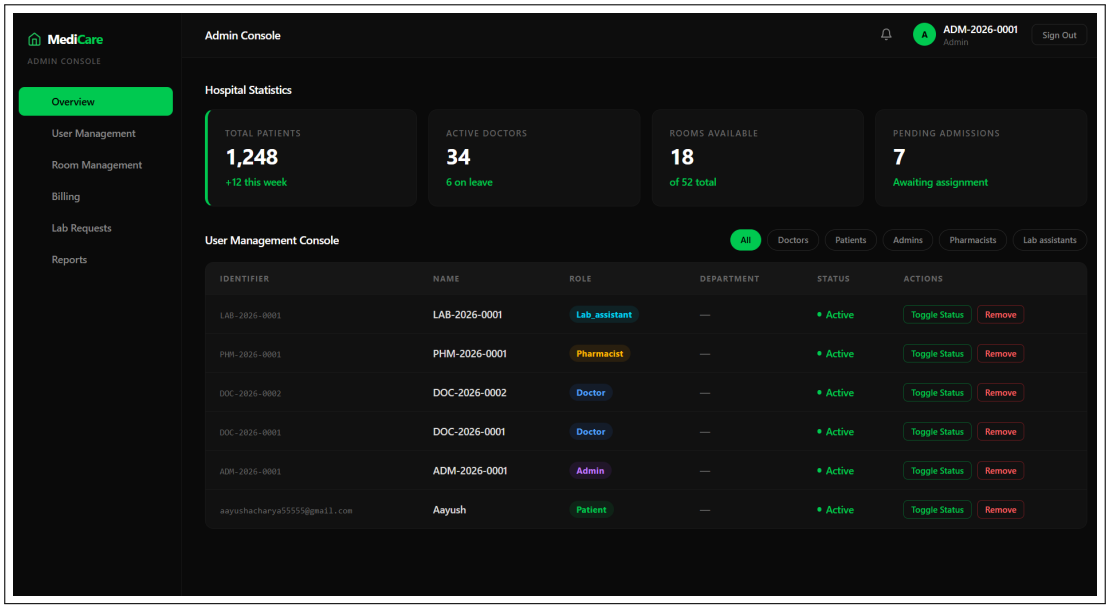

Figure 4.5: Admin dashboard

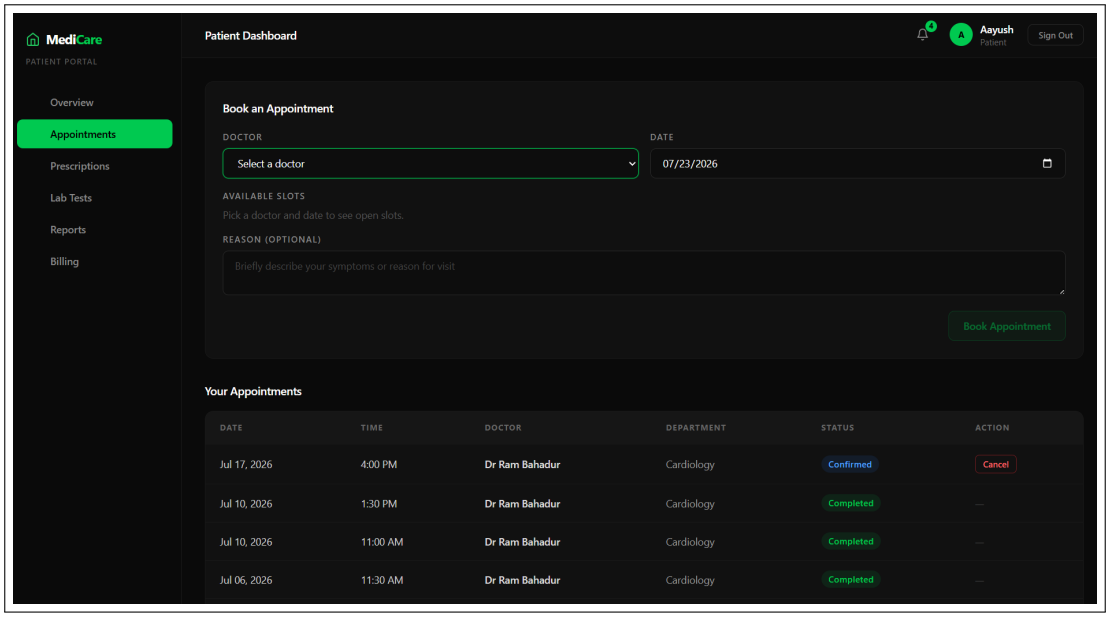

Figure 4.6: Appointment booking and management

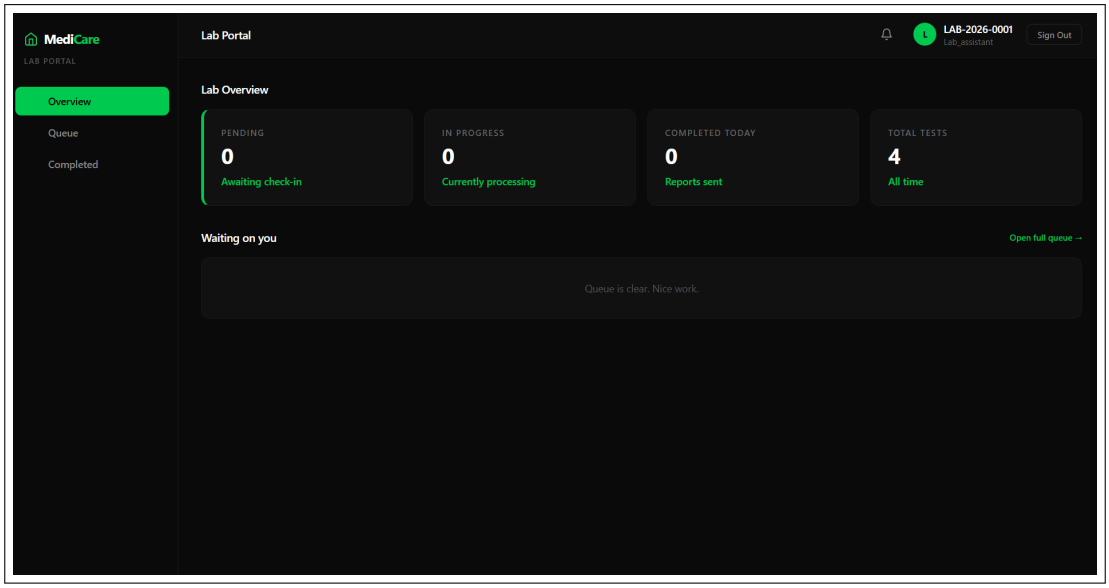

Figure 4.7: Lab Assistant portal — test-request queue and results

Billing and Pharmacy Inventory. Administrators issue invoices and record payments; pharmacists manage a location-aware medicine catalogue with stock levels, reorder thresholds, and expiry.

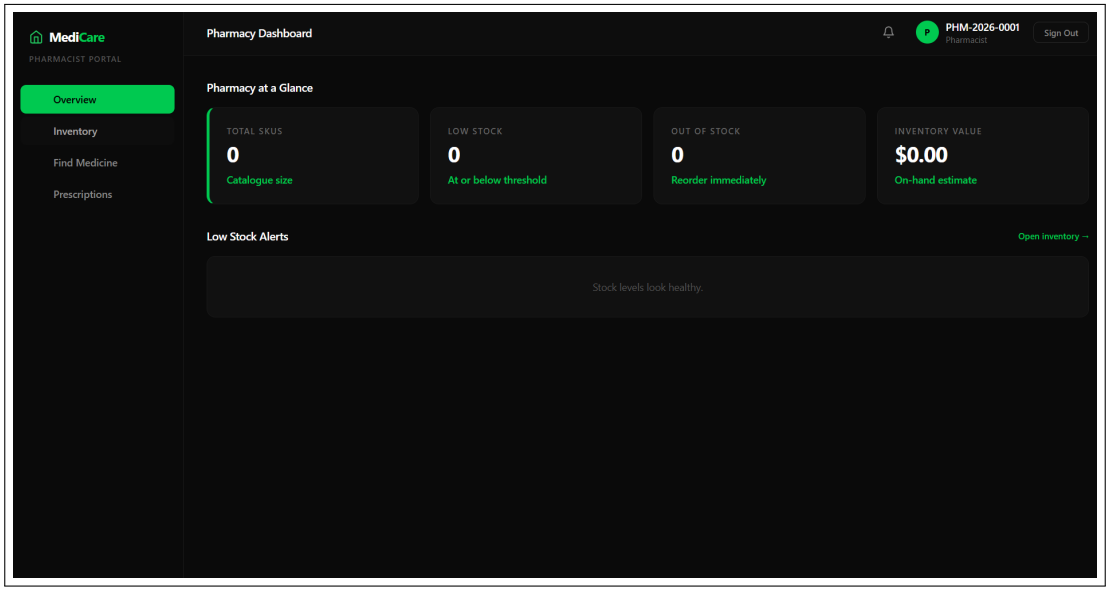

Figure 4.8: Pharmacist portal — location-aware medicine inventory

Notifications. In-application notifications are created for appointments, prescriptions, lab tests, referrals, and invoices, and are mirrored to the user's email (and optionally SMS), so patients are informed without logging in.

4.2 Results and Performance Analysis

The system was tested on a standard personal computer with a local MySQL server. The backend started successfully, established a database connection, and synchronised all thirteen models; the frontend built and served without errors; and the REST endpoints returned correct responses for authentication, booking, and billing flows. Typical API responses completed well within the 2–3 second target under normal single-user load.

4.3 Comparison with Project Objectives

The implemented system meets the objectives stated in Chapter 1:

- **Web-Based Design:** a full-stack architecture (React + Node.js/Express + MySQL) is in place.
- **Secure Authentication:** JWT with refresh-token rotation, password reset, and email verification secure access, with five-way role-based control.
- **Workflow Automation:** appointments, prescriptions, lab tests, referrals, billing, and pharmacy inventory are all implemented.
- **Multi-Channel Notifications:** in-app, email, and SMS notifications keep users informed.
- **Usability:** responsive, role-specific dashboards provide an intuitive interface.

4.4 Challenges and Limitations

- **Schema Evolution:** during development the schema is synchronised automatically; a production deployment would need managed migrations.
- **Concurrency:** some multi-step operations (for example, recording payments) would benefit from database transactions to remain fully atomic under concurrent load.
- **External Delivery:** real email/SMS delivery depends on third-party providers being configured; without them the system falls back to console logging.
- **No Mobile Application:** the system is currently web-only.
- **Automated Testing:** the current version relies on manual verification rather than an automated test suite.

4.5 Discussion

The Hospital Management System demonstrates effective integration of frontend and backend technologies. Axios and RESTful APIs provide smooth communication between the React frontend and the Express backend; JWT-based authentication with refresh-token rotation provides secure, revocable access; the layered architecture separates presentation, application, and data concerns; and the normalised MySQL schema enforces referential integrity across clinical and administrative records. The modular design makes it straightforward to add further features.

4.6 Summary

The implemented system achieves its primary objectives of secure authentication, role-based access, automated hospital workflows, and multi-channel notification. While some limitations remain, the system provides a solid foundation for future enhancement and a practical tool for managing hospital operations.

Chapter 5

Conclusion and Future Works

5.1 Conclusion

The Hospital Management System developed in this project successfully fulfils the primary objectives outlined in Chapter 1 by providing a secure, web-based platform for managing hospital operations across five distinct roles. The system implements JWT-based authentication with refresh-token rotation to ensure secure, revocable access and enforces role-based authorisation, while dedicated dashboards for patients, doctors, administrators, lab assistants, and pharmacists enhance usability. Patients can book and manage appointments, view prescriptions and lab results, and settle bills; doctors manage availability, prescriptions, lab orders, and referrals; and administrators manage staff, billing, and rooms. Appointments, prescriptions, lab tests, referrals, and invoices are handled with automatic status updates and notifications delivered both in-app and by email. The frontend, built with React and Tailwind CSS, offers a responsive and user-friendly interface, while the modular client–server architecture combined with MySQL and Sequelize ensures maintainability and flexibility for future expansion. Overall, the project demonstrates the effective integration of modern frontend and backend technologies, secure authentication, and reliable relational data management.

5.2 Limitations

While the system is fully functional, a few limitations exist in the current version:

- **Managed Migrations:** the schema is auto-synchronised in development rather than versioned through migrations.
- **Transactions:** some multi-step flows are not yet fully transactional.
- **Provider Dependency:** outbound email and SMS require external providers to be configured.

- **Mobile Support:** the system is currently web-only.

5.3 Future Works

The modular and scalable architecture allows for multiple future enhancements:

- **Database Migrations and Transactions:** adopt versioned migrations and wrap critical flows (payments, booking) in transactions.
- **Automated Testing and CI:** add unit and integration tests with continuous integration.
- **Advanced Scheduling and Analytics:** smarter slot allocation, queue management, and dashboards tracking hospital usage and trends.
- **Mobile Application:** a companion app for Android and iOS.
- **Inpatient and Pharmacy Extensions:** ward/bed management and automatic stock deduction on dispensing.
- **Audit Logging and Compliance:** record access to medical data to support privacy and compliance requirements.

The system is designed with future scalability in mind, allowing it to evolve into a comprehensive hospital management solution suitable for modern healthcare institutions.

Bibliography

[1] React: Official documentation. https://react.dev/. Accessed July 27, 2026.

[2] Node.js: Official documentation. https://nodejs.org/en/docs/. Accessed July 27, 2026.

[3] Express.js: Official guide. https://expressjs.com/. Accessed July 27, 2026.

[4] MySQL: Official documentation. https://dev.mysql.com/doc/. Accessed July 27, 2026.

[5] Sequelize: ORM documentation. https://sequelize.org/docs/v6/. Accessed July 27, 2026.

[6] JSON Web Tokens: Introduction. https://jwt.io/introduction/. Accessed July 27, 2026.

[7] Tailwind CSS: Documentation. https://tailwindcss.com/docs. Accessed July 27, 2026.

[8] Axios: Promise-based HTTP client. https://axios-http.com/docs/intro. Accessed July 27, 2026.

[9] Nodemailer: Documentation. https://nodemailer.com/. Accessed July 27, 2026.

[10] OpenMRS: Open-source medical record platform. https://openmrs.org/. Accessed July 27, 2026.

Appendix-A

A.1 Source Code Snippets

A.1.1 Backend: User Model (User.js)

The following snippet shows the Sequelize model for the `users` table, including the role enumeration, refresh-token version used for session revocation, and the email-verification timestamp.

```
const User = sequelize.define("User", {
  id: { type: DataTypes.UUID, defaultValue: DataTypes.UUIDV4,
     primaryKey: true },
  identifier: { type: DataTypes.STRING(100), allowNull: false, unique:
     true },
  password_hash: { type: DataTypes.STRING(255), allowNull: false },
  role: {
    type: DataTypes.ENUM("patient","doctor","pharmacist","lab_assistant
       ","admin"),
    allowNull: false,
  },
  refresh_token_version: { type: DataTypes.INTEGER, defaultValue: 0 },
  email_verified_at: { type: DataTypes.DATE, allowNull: true },
  is_active: { type: DataTypes.BOOLEAN, defaultValue: true },
}, { tableName: "users", timestamps: true });
```

A.1.2 Backend: Role-Based Access Middleware (auth.js)

This middleware verifies the JWT and restricts a route to specific roles.

```
export const authenticate = async (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith("Bearer"))
    return res.status(401).json({ message: "No token provided" });
  const decoded = verifyAccessToken(authHeader.split(" ")[1]);
  const user = await User.findByPk(decoded.id,
    { attributes: { exclude: ["password_hash"] } });
  if (!user || !user.is_active)
```

```
    return res.status(401).json({ message: "User not found or inactive"
        });
  req.user = user;
  next();
};

export const authorise = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user?.role))
    return res.status(403).json({ message: "Access denied" });
  next();
};
```

A.2 API Endpoints Summary

The table below summarises the main API endpoints (base path `/api/v1/hms`).

Endpoint	Method	Description
`/register`	POST	Register a new patient
`/login`	POST	Log in an existing user
`/refresh`	POST	Refresh the access token
`/forgot-password`	POST	Request a password-reset link
`/reset-password`	POST	Set a new password from a token
`/verify-email`	POST	Verify an email address
`/staff`	POST	Create a staff account (admin)
`/appointments`	POST	Book an appointment
`/prescriptions`	POST	Issue a prescription
`/labs`	POST	Request a lab test
`/invoices`	POST	Create an invoice (admin)
`/medicines`	GET	List the medicine catalogue

Table 1: Main API Endpoints of the HMS

A.3 Configuration Files

Example environment variables (`.env`) storing sensitive configuration such as the database connection, JWT secrets, and optional mail/SMS providers:

```
PORT=8000
NODE_ENV=development
```

```
CLIENT_URL=http://localhost:5173

DB_HOST=localhost
DB_PORT=3306
DB_NAME=hms_db
DB_USER=root
DB_PASSWORD=your_password

JWT_ACCESS_SECRET=your_access_secret
JWT_REFRESH_SECRET=your_refresh_secret

# Optional: outbound email (falls back to console when unset)
MAIL_HOST=
MAIL_PORT=587
MAIL_FROM=no-reply@dhulikhel-hms.local

# Optional: outbound SMS (falls back to console when unset)
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_FROM_NUMBER=
```

A.4 Project Timeline: Gantt Chart

The Gantt chart in Figure 1 illustrates the timeline of major tasks and milestones for the HMS project over eight weeks.

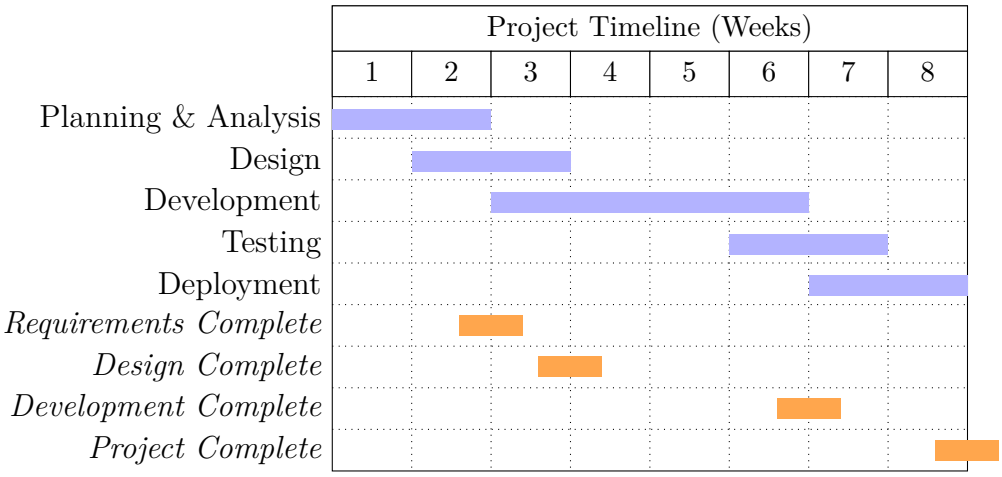

Figure 1: Eight-Week Project Timeline — Major Phases